MAIE 6000C: Engineering AI Products: From Prototype to Production

Week 6 — Lab

Program: MSc in AI & Entrepreneurship

Term: Fall 2026/27

Instructor: Lefteris NTAFLOS, PhD

The Hong Kong University of Science and Technology

Lab Session 1 — Observe the Starter Worker System

Session 1 Plan and Team Roles

Session 1 plan

- Start the stack with the worker stopped
- Create a case and inspect its initial state
- Start the worker and observe state changes
- Trace route → job → worker → result → API
- Inspect tests and, if safe, observe failure behavior
- Build a worker-lifecycle evidence map

Suggested roles

- **Driver:** runs commands
- **Evidence recorder:** records paths, functions, states, outputs
- **Skeptic:** asks “How do we know?”
- **Test owner:** finds and runs relevant tests

Today’s Rule: Observe the Lifecycle Before You Extend It

Worker path = more than a container

```
API creates record and job
          ↓
job waits in durable state
          ↓
worker claims and processes work
          ↓
result / retry / failure is persisted
          ↓
API exposes current state
```

Record evidence separately

Evidence type	What it proves
HTTP response	What a caller can observe
Code	Where behavior is implemented
Database state	What persisted
Logs	Execution detail
Tests	What behavior is verified

Start the Starter System

```
# 1. Open docker desktop
# 2. Open Powershell / Bash Terminal
# 3. cd into working folder
docker compose up --build -d
docker compose ps
docker compose config --services
docker compose stop worker
# can also keep API Swagger page open:
http://localhost:8000/docs
```

```
# PowerShell
curl.exe -i
http://localhost:8000/health/live
curl.exe -i
http://localhost:8000/health/ready


# Bash: remove .exe
curl -i http://localhost:8000/health/live
curl -i http://localhost:8000/health/ready
```

Why stop the worker?

So the team can observe the honest initial queued or pending state before processing begins.

Locate the Actual Lifecycle Before Guessing

Find the starter system’s real implementation

Known Week 5 anchors:

```
services/common/schemas.py
services/common/models.py
services/api/app/main.py
tests/integration/test_api_case_flow.py
```

Find evidence for:

- worker entrypoint;
- job status fields;
- claim logic;
- AI-service call;
- success persistence;
- failure or retry handling;
- worker-related tests.

Repository searches:

```
git grep -n "create_case_with_job" -- services
git grep -n "class Job" -- services
git grep -n "status" -- services
git grep -n "attempt" -- services
git grep -n "claim" -- services
git grep -n "retry" -- services
git grep -n "worker" -- services tests
git ls-files services tests
```

Create a Case While the Worker Is Stopped

Create one valid case (adjust the port to 8000 or 8001)

```powershell
# PowerShell
$body = @{
  title = "Worker lifecycle observation"
  description = "Use this case to observe durable job creation and background processing."
} | ConvertTo-Json

$created = Invoke-RestMethod `
  -Uri "http://localhost:8001/cases" `
  -Method Post `
  -ContentType "application/json" `
  -Body $body

$created | ConvertTo-Json -Depth 10
```

```bash
# Bash
curl -sS -X POST "http://localhost:8000/cases" \
  -H "Content-Type: application/json" \
  -d '{"title":"Worker lifecycle observation","description":"Use this case to observe durable job creation and background processing."}' \
  | python -m json.tool
```

Record:

- HTTP status;
- case ID;
- job ID;
- initial case status;
- initial job status;
- response shape.

Open Swagger at

- http://localhost:8000/docs

- `GET /cases/{case_id}`
- `GET /jobs/{job_id}`

Start the Worker and Observe State Changes

Watch the workflow cross the worker boundary

Terminal 1 — Worker logs

```
docker compose logs -f --tail=50 worker
```

Terminal 3 — Start the worker

```
docker compose start worker
```

Optional

```
docker compose logs --tail=30 ai
```

Terminal 2 — Poll the resource

```
# PowerShell
while ($true) {
  curl.exe -s http://localhost:8000/cases/CASE_ID
  echo “”
  Start-Sleep -Seconds 1
}

# Bash
while true; do
  curl -s http://localhost:8000/cases/CASE_ID
  echo
  sleep 1
done
```

Compare API State, Database State, and Logs

Different evidence sources answer different questions

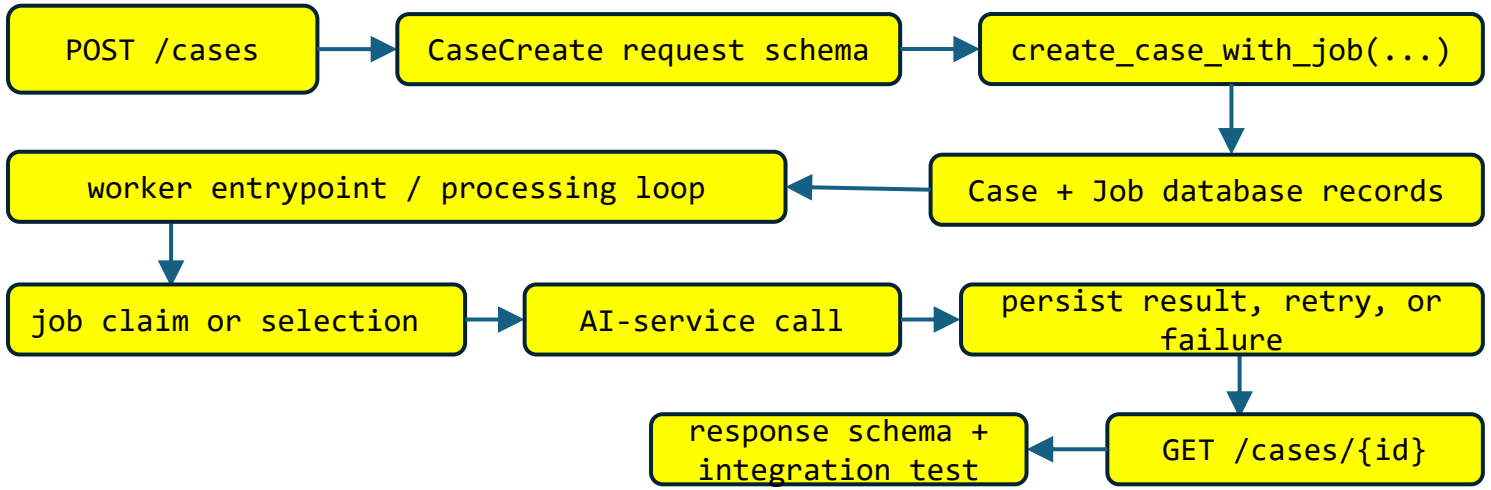
API GET	Database record	Worker log
What can the client see now?	What state and result persisted?	What happened during execution?

Inspect your database:

```
docker compose exec db psql -U postgres -d maie6000c -c 'SELECT * FROM cases;’
docker compose exec db psql -U postgres -d maie6000c -c 'SELECT * FROM jobs;’
```

Trace One Worker Path Through the Repository

Follow one request as a connected system



Questions to answer:

- Are case and job created together?
- Where is processing set?
- How is work claimed?
- Where is attempt count changed?
- What happens after AI success?
- What happens after AI failure?
- How does the final state reach the API response?

Activity A — Worker Lifecycle Evidence Map – 15min

For each row, record exact evidence.

Question	Actual evidence	Path / symbol / command	Gap or question
What API route begins the workflow?			
Where are resource and job created?			
What is the initial state?			
What process claims the job?			
What calls the AI-enabled component?			
What persists the result?			
What happens on failure or timeout?			
Which test proves part of this flow?			

Lab Session 2 — Design and Begin the Team Worker Path

From Starter Evidence to Your Team's System

Session 2 goal

Starter evidence

↓

One project-specific async workflow

↓

Resource + job lifecycle

↓

Failure and retry decisions

↓

Week 7 demo and test plan

Outcome

- Each team leaves with a worker path that is:
- meaningful;
- small enough for Week 7;
- visible through the API;
- testable;
- documented in the real repository.

Activity B: Choose Work That Is Actually Asynchronous – 5min

Select one workflow step for the worker

Good worker candidates:

- AI classification, summarization, recommendation;
- document parsing or OCR;
- long-running calculation;
- external-service call;
- batch generation or scheduled reprocessing.

Candidate step	Long / uncertain?	Network or AI call?	Retryable?	User needs final result immediately?	Decision
Example: analyze submission	Yes	Yes	Yes	No	Worker
Example: validate title	No	No	No	Yes	API
Your project step					

Scope the Minimum Week 7 Worker Path

Minimum viable worker path

```
POST /<resource>
  → resource + job created
  → initial state returned
```

```
Worker
  → processes one meaningful task
  → persists one useful result or one
meaningful failure
```

```
GET /<resource>/{id}
  → exposes current state and
result/failure
```

Usually out of scope before Week 7

- multiple workflow types;
- large-scale worker pools;
- full scheduling platform;
- dashboards;
- complex role systems;
- advanced distributed guarantees;
- many optional endpoints.

Activity C: Design the Resource and Job Lifecycle – 7min

Complete this state-ownership table

State	Owner	Entry condition	Persisted evidence	API-visible meaning	Next state(s)
queued	API	valid create request	resource + job rows	accepted, waiting	processing
processing	Worker	job safely claimed	claim / timestamp / worker marker	work is underway	completed, retry_wait, failed
retry_wait	Worker	temporary failure	attempt count + next retry time	delayed retry	queued, failed
completed	Worker	result persisted	result fields	final result available	—
failed	Worker	retry budget exhausted / non-retryable error	safe failure code	processing did not complete	optional explicit retry

Use your own project nouns and state names where appropriate.

Activity D: Define Boundaries and State Ownership - 5min

Decide who may do what

Discuss:

- Which transition does the API make?
- Which transitions belong only to the worker?
- Can a user request a retry?
- What authorization scope is realistic for your project?

Component	May do	Must not do
Client	submit work; retrieve status	directly mark processing/completed
API	validate; authorize; create resource/job	wait indefinitely for AI result
Worker	claim job; process; persist state/result	expose public client API directly
AI component	receive limited input; return result/error	directly control client-facing state
Scheduler, optional	enqueue eligible jobs	bypass persistent lifecycle

Activity E: Failure, Retry, and Idempotency Decisions – 6min

Build a failure policy before implementation

- **Required team decision**
- Choose one:
  - idempotency required for Week 7;
  - planned for later;
  - explicitly out of scope, with risk documented.

Scenario	Retry?	Maximum attempts / delay	Client-visible state	Test
Invalid create request	No	—	422	validation test
AI timeout	Yes / No	define policy	retry_wait or failed	timeout test
AI returns malformed result	Usually no	—	failed	processor test
Worker crash	Recover	define recovery rule	queued / processing recovery	recovery test or documented gap
Client repeats POST	idempotency decision	define scope	original or duplicate behavior	create/retry test

Activity F: Write the Worker Contract and Pseudocode – 5min

Define the worker before writing scattered code

```
Worker input:

- job ID or safely claimed job

- resource loaded from durable storage


Worker steps:

1. claim one eligible job atomically

2. mark processing

3. load required resource data

4. call processor with finite timeout

5. validate output

6. persist result + completed state

7. on retryable failure: schedule retry

8. on terminal failure: persist safe failed state
```

Document:

- timeout;
- input data sent to AI component;
- expected output;
- result persistence;
- retry policy;
- safe failure code.

Activity G: Build the Week 7 Test and Demo Plan – 6min

Testing rule

- Poll for a terminal state with a deadline. Do not rely on a fixed long sleep().

Minimum Week 7 tests

Test name	Setup and request	Expected HTTP result	Expected persisted state
`test_create_<resource>_creates_queued_job`	valid POST	201 or documented success	resource + queued job
`test_worker_completes_<resource>`	create + run worker/fake processor	GET returns completed result	result saved; job completed
`test_invalid_<resource>_returns_422`	missing/invalid input	422	no unintended job
`test_processor_failure_is_visible`	fake timeout/error	GET shows retry or failed state	failure safely persisted

Repository Sprint: Save the Real Engineering Decisions

Update real repository artifacts now

Minimum recommended updates:

- `docs/architecture.md`

Add or refine:

- worker-path diagram;
- resource and job lifecycle;
- state ownership;
- timeout and retry policy;
- worker/AI boundary;
- Week 7 demo path;
- Week 7 test plan.

Optional supporting document:

- `docs/async-workflow.md`

Team sprint roles:

1. implementation owner;
2. documentation owner;
3. test owner;
4. integration / Compose owner.

If generative AI assisted, add the required AI Use Statement.